SE2030 – Software Engineering

Lab Sheet 03 – Use Case Diagram

# Web-Based Library Management System

Use Case Report

## Group 2

| Student ID | Name | Assigned Function |
|---|---|---|
| IT21047756 | Weerasekara W.B.M.K.D | Return Management |
| IT21054372 | Samarasinghe P.D.P | Book Management |
| IT21060144 | Rathnayake R.M.I | Reservation Management |
| IT21077524 | Pathirana A.P.C.E | User Management |
| IT21101274 | Dolawatta T.Y.R | Fine Management |
| IT21158872 | Vimukthi W.M.B.V | Borrowing Management |

Sri Lanka Institute of Information Technology (SLIIT)

Faculty of Computing | Year 2 Semester 1 | 2026

## 1. Introduction

This report is prepared for the Lab Sheet 03 task of SE2030 Software Engineering. Our group project is a Web-Based Library Management System for a university library. The system is meant to replace manual library processes with a simple web platform that lets librarians, students and lecturers manage books and borrowing online.

In this report we have included the Use Case Diagram for our system and one use case scenario written by each group member based on their assigned function.

## 2. Actors

These are the people who will use the system. Each actor has a different role and different things they can do.

| Actor | Role in the System |
| --- | --- |
| Administrator | Manages user accounts and system settings. Has full access to everything. |
| Senior Librarian | Handles issuing and returning books. Manages daily library transactions. |
| Assistant Librarian | Adds and updates book records in the database. |
| Library Staff | Helps with returns and updating information. |
| Undergraduate Student | Searches for books, borrows, returns and reserves books. |
| Lecturer | Borrows and reserves books for teaching and research. |
| Student Services Officer | Deals with student fines and borrowing history queries. |

## 3. Use Cases

The table below lists all the main use cases we identified for the system.

| ID | Use Case | Actor(s) |
| --- | --- | --- |
| UC-01 | Manage Users | Administrator |
| UC-02 | Manage Books | Senior Librarian, Assistant Librarian |
| UC-03 | Manage Borrowing | Senior Librarian, Student, Lecturer |
| UC-04 | Manage Returns | Senior Librarian, Library Staff |
| UC-05 | Manage Reservations | Student, Lecturer |
| UC-06 | Manage Fines | Student Services Officer, Librarian |
| UC-07 | Search Books | Student, Lecturer |
| UC-08 | User Login / Logout | All actors |
| UC-09 | Extend Borrow Period | Senior Librarian |
| UC-10 | Generate Reports | Administrator |

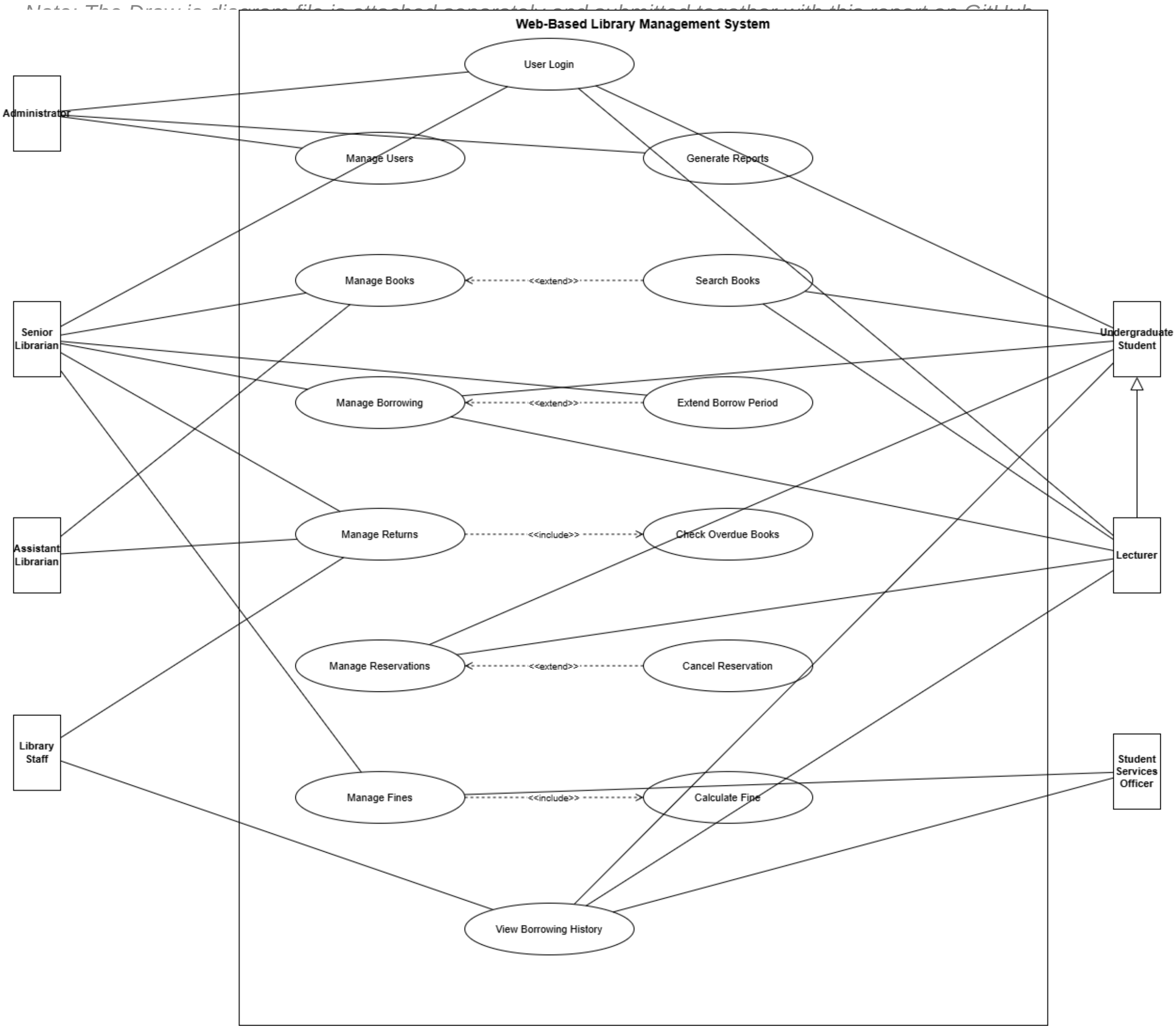
# 4. Use Case Diagram

The diagram below shows how each actor interacts with the system. We used Draw.io to draw it as instructed in the lab sheet.

Relationships used in the diagram:

- Association – a straight line connecting an actor to a use case they use directly.
- «extend» – a dashed arrow showing an optional action (e.g. Extend Borrow Period extends Manage Borrowing).
- «include» – a dashed arrow showing a step that always happens (e.g. Calculate Fine is always included in Manage Fines).
- Generalization – Registered Student has access to everything a normal User can do plus more.

*Note: The Draw.io diagram file is attached separately and submitted together with this report on GitHub.*



## 5. Use Case Scenarios

Each group member has written a use case scenario for the function they were assigned. The scenarios describe step by step how users interact with that part of the system.

| UC-01 – User Management | |
|---|---|
| **Actors** | Administrator (primary), Senior Librarian (secondary) |
| **Description** | This use case is about managing user accounts in the library system. The admin can add new users, see their details, change their information, or remove them when they are no longer part of the university. This covers students, lecturers and library staff accounts. |
| **Pre-condition** | Admin must be logged in to the system. |
| **Post-condition** | User account is created, updated or removed from the system. |
| **Main Flow** | 1. Admin logs in using username and password.<br>2. Admin goes to the User Management section.<br>3. System shows the list of all users.<br>4. Admin picks what to do – add, view, edit or delete a user.<br>5. If adding: admin fills the form with the new user's details and submits.<br>6. System saves the new user and shows a success message.<br>7. If editing: admin changes the needed fields and saves.<br>8. If deleting: admin selects the user, confirms, and the record is removed.<br>9. Use case ends. |
| **Alternative Flow** | - If the student ID already exists, the system shows an error saying the user already exists.<br>- If required fields are left empty, system will not save and asks admin to fill them.<br>- If the user still has books borrowed, system will not allow deletion until books are returned. |

| UC-02 – Book Management | |
|---|---|
| **Actors** | Senior Librarian (primary), Assistant Librarian (secondary) |
| **Description** | This use case handles managing books in the library. Librarians can add new books to the system, update information like the title or author, check which books are available, and remove books that the library no longer has. Keeping the book database accurate is the main goal here. |
| **Pre-condition** | Librarian must be logged in. |
| **Post-condition** | Book record is added, changed, or removed from the database. |
| **Main Flow** | 1. Librarian logs in to the system.<br>2. Goes to the Book Management page.<br>3. System displays the current list of books in the library.<br>4. Librarian chooses an action – add, search, edit or remove a book.<br>5. To add: librarian enters book details like ISBN, title, author, and number of copies.<br>6. System checks the details and saves the new book record.<br>7. To edit: librarian finds the book, makes changes and saves.<br>8. To remove: librarian selects the book, confirms and it gets deleted.<br>9. System shows a success message and the list is updated.<br>10. Use case ends. |
| **Alternative Flow** | - If the ISBN entered already exists in the system, an error message is shown.<br>- If a required field like title or author is missing, the form will not submit.<br>- A book cannot be deleted if it is currently borrowed by someone. |

| UC-03 – Borrowing Management | |
|---|---|
| **Actors** | Senior Librarian (primary), Student / Lecturer (secondary) |
| **Description** | When a student or lecturer wants to take a book from the library, the librarian records it through this use case. The system keeps track of who borrowed what, when it was taken, and when it needs to come back. The librarian can also extend the borrow period or cancel a record if needed. |
| **Pre-condition** | User must be logged in. Book must be available in the library. |
| **Post-condition** | Borrow record is saved. Book availability count goes down by one. |
| **Main Flow** | 1. Librarian logs in and opens the Borrowing Management section.<br>2. Librarian enters the user ID and book ISBN.<br>3. System checks if the book is available and if the user is allowed to borrow.<br>4. System shows the book and user details for the librarian to confirm.<br>5. Librarian confirms and the system records the borrow transaction.<br>6. System sets the issue date and due date automatically.<br>7. Book availability in the system is reduced by one.<br>8. A confirmation message is shown.<br>9. Use case ends. |
| **Alternative Flow** | - If the book has no available copies, system suggests the user make a reservation instead.<br>- If the user has already reached their borrow limit, the transaction is blocked.<br>- If the user has unpaid fines, the librarian sees a warning before proceeding. |

| UC-04 – Return Management | |
|---|---|
| **Actors** | Senior Librarian (primary), Library Staff (secondary) |
| **Description** | This use case is for when a student or lecturer brings back a book to the library. The librarian records the return in the system, and the book becomes available again for others. If the book is returned late, the system will automatically calculate the fine based on how many days overdue it is. |
| **Pre-condition** | There must be an active borrow record for the book being returned. |
| **Post-condition** | Return is recorded, book availability is updated, fine is calculated if overdue. |
| **Main Flow** | 1. Librarian logs in and opens the Return Management section.<br>2. Librarian enters the book ISBN.<br>3. System finds the active borrow record for that book.<br>4. System shows who borrowed it and the due date.<br>5. Librarian confirms the return.<br>6. System marks the borrow record as returned.<br>7. Book availability count goes up by one.<br>8. System checks if the return is late.<br>9. If late, fine is calculated (number of late days x Rs.10) and saved.<br>10. Confirmation message is shown.<br>11. Use case ends. |
| **Alternative Flow** | - If no borrow record is found for the book, system shows an error and asks to check the ISBN.<br>- If the book is returned on time, no fine is added.<br>- If another user has reserved that book, the system notifies them that it is now available. |

| UC-05 – Reservation Management | |
|---|---|
| **Actors** | Student / Lecturer (primary), Senior Librarian (secondary) |
| **Description** | Sometimes a book a student needs is already borrowed by someone else. In that case they can use this feature to reserve it. Once the book is returned, the system will let the next person in the queue know it is available. Students can also cancel or change their reservation if they no longer need it. |
| **Pre-condition** | User must be logged in. The book they want must currently be unavailable. |
| **Post-condition** | Reservation is saved and the user is added to the waiting list. |
| **Main Flow** | 1. User logs in and searches for a book.<br>2. System shows the book is currently not available.<br>3. User clicks Reserve.<br>4. System checks the user does not already have a reservation for this book.<br>5. System creates a reservation record with the user details and date.<br>6. User is assigned a position in the waiting queue.<br>7. System confirms the reservation.<br>8. When the book is returned, system sends a notification to the first person in the queue.<br>9. Use case ends. |
| **Alternative Flow** | - If the user already reserved the same book before, system will not allow a duplicate.<br>- If the book becomes available before the reservation is confirmed, the user is told to borrow it directly.<br>- If user wants to cancel, they can do so and the queue shifts up for others waiting. |

| UC-06 – Fine Management | |
|---|---|
| **Actors** | Student Services Officer (primary), Senior Librarian (secondary) |
| **Description** | This use case handles the fines that students get when they return books late. The system calculates the fine automatically when a late return happens. The formula is simple: Fine = number of late days multiplied by Rs.10 per day. For example if a book was due on June 14 but returned on June 16, the fine is Rs.20. The Student Services Officer can view fines, mark them as paid, or remove wrong entries. |
| **Pre-condition** | There must be an overdue borrow record in the system. |
| **Post-condition** | Fine record is created and payment status is tracked. |
| **Main Flow** | 1. When a book is returned late, system automatically calculates the fine.<br>2. Fine record is created and linked to the student's account.<br>3. Student Services Officer logs in and opens Fine Management.<br>4. Officer views the list of all outstanding fines.<br>5. Officer selects a fine to view the details.<br>6. Student pays the fine.<br>7. Officer marks the fine as paid in the system.<br>8. System updates the status to Paid.<br>9. Confirmation is shown.<br>10. Use case ends. |
| **Alternative Flow** | - If a fine record was created by mistake, the officer can delete it after checking.<br>- If payment is done in parts, officer records the partial payment and the fine stays open.<br>- If the student disputes the fine, the librarian can review the borrow record and correct it if needed. |

## 6. Conclusion

In this lab we identified the actors and use cases for our library management system and drew a use case diagram using Draw.io. We also each wrote a use case scenario for the function we were assigned in the group project.

Going through this process helped us understand what the system needs to do from the user's point of view. It also helped us see how the different parts of the system are connected to each other. We will use this as a base when we move on to designing the system in the next phases.